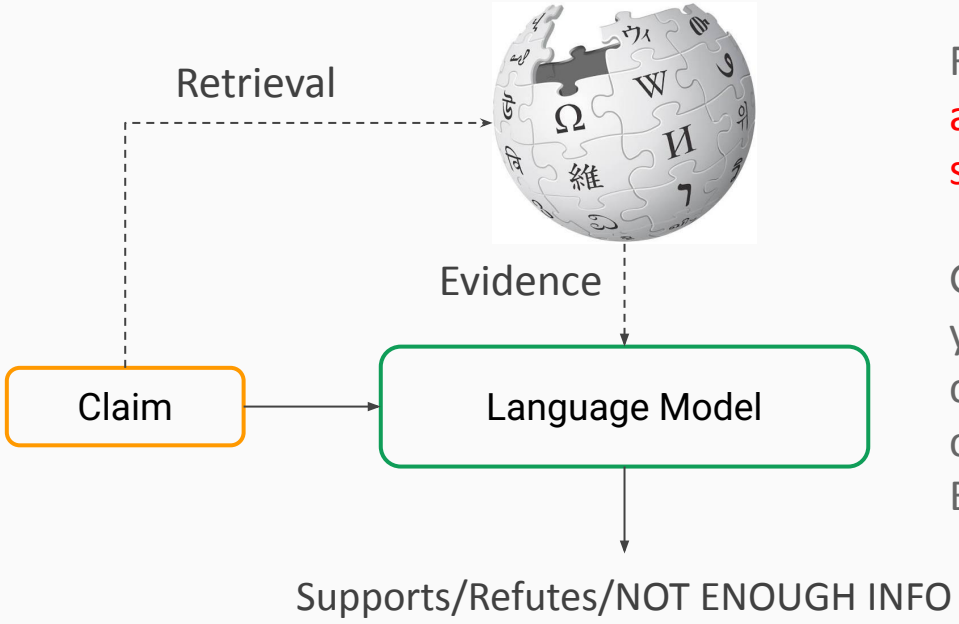
Assignment 3: Fact Verification

NTHU Natural Language Processing

Hung-Yu Kao

IKM Lab TAs

Fact Verification



Fact Verification aims to determine whether a claim is supported, refuted, or lacks sufficient evidence.

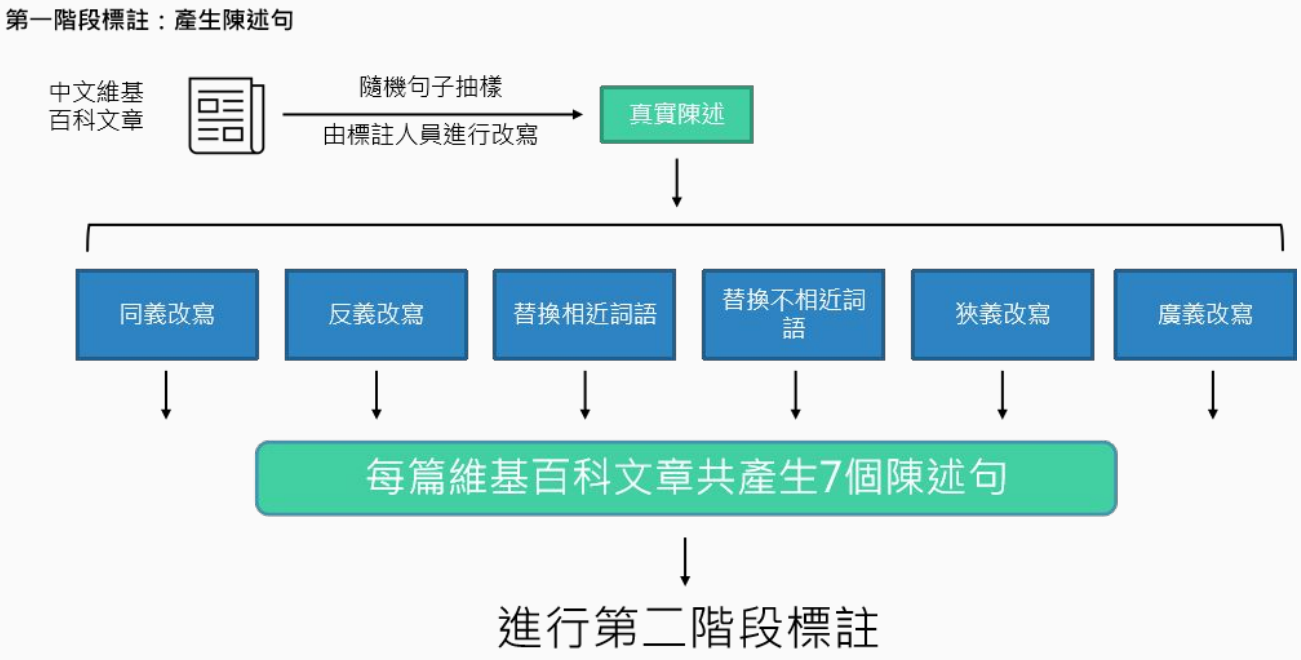
Given a claim and Wikipedia documents, you must retrieve the correct evidences and classify the claim into one of three categories: Supports, Refutes, or Not Enough Information (NEI).

Assignment Description

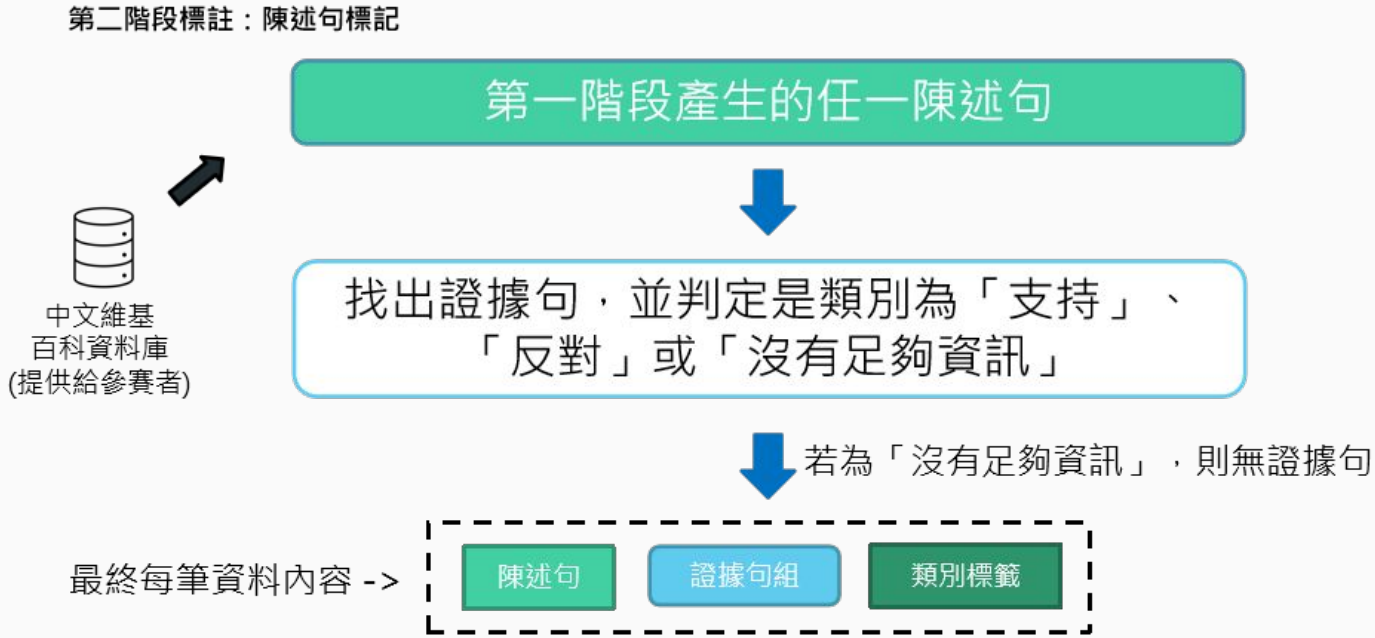
- In Assignment 3, you will practice building a fact verification system that integrates evidence retrieval and claim classification.

- You will use the CFEVER dataset to retrieve relevant evidence from Wikipedia and classify each claim as Supports, Refutes, or Not Enough Information (NEI).

CFEVER dataset



CFEVER dataset



Dataset

*CFEVER dataset

- Train split: 24012 claims
- Development split: 3000 claims
- Test split: 6000 claims
- Each dat: A claim, several evidence, a label

[1] CFEVER: A Chinese Fact Extraction and VERification Dataset

Data Example (CFEVER)

Each example contains a claim and multiple of evidences.

The label indicates whether the evidence supports, refutes, or provides insufficient information to verify the claim.

Claim: 尼克・貝爾格在 2004 年前往中東尋找生意商機。(Nick Berg traveled to the Middle East in 2004 looking for business opportunities.)

Evidence:

[wiki/尼克・貝爾格 (Nick Berg)]

他在 2004 年 3 月前往伊拉克 尋找通訊生意商機，自 2004 年 4 月 9 日失蹤。(He went to Iraq in March 2004 to search for communication business opportunities and has been missing since April 9, 2004.)

[wiki/伊拉克 (Iraq)]

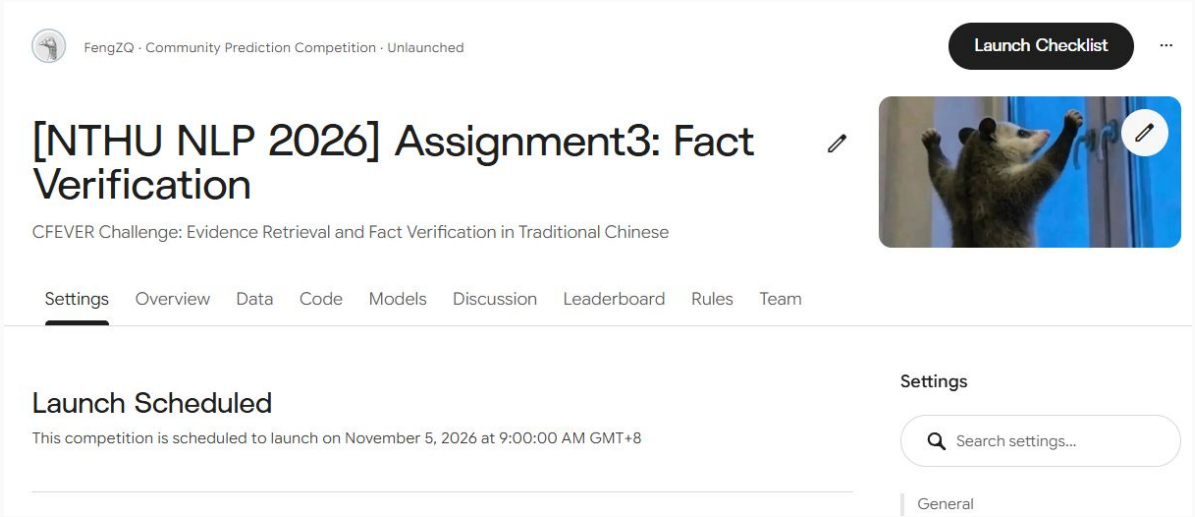
伊拉克共和國，通稱伊拉克，位於西亞—中東地區的共和國。(The Republic of Iraq, commonly known as Iraq, is a republic located in West Asia - the Middle East.)

Verdict: Supports

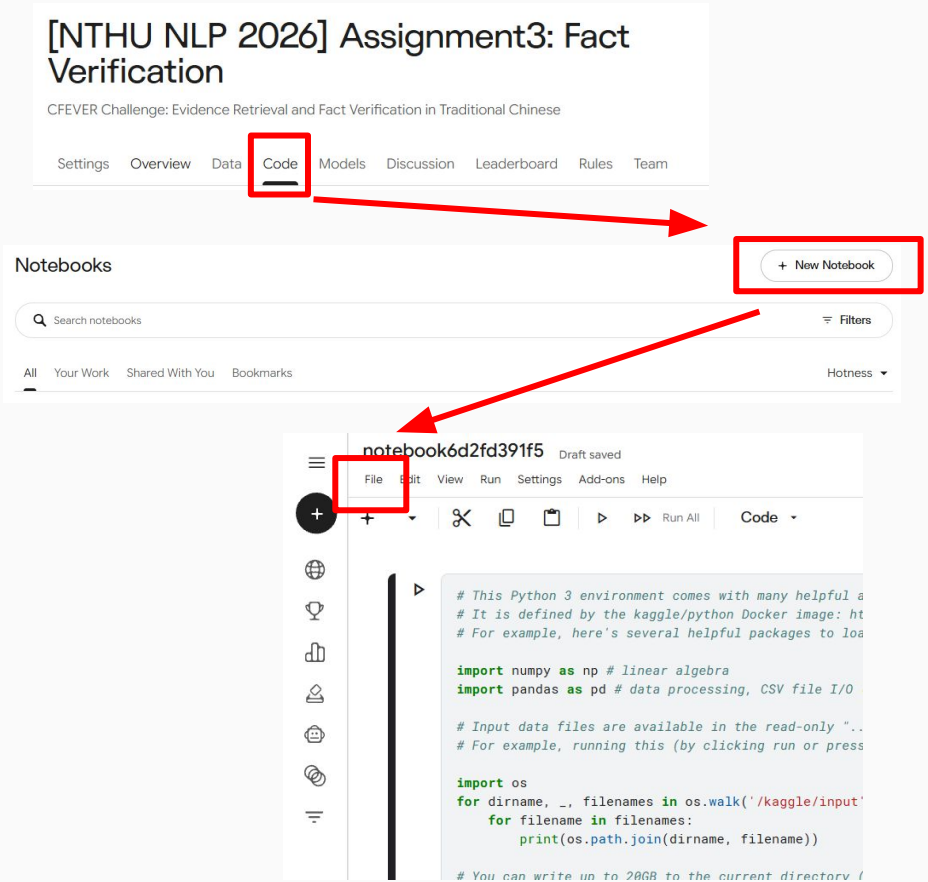
[AAAI 2024] CFEVER: A Chinese Fact Extraction and VERification Dataset

Kaggle Competition

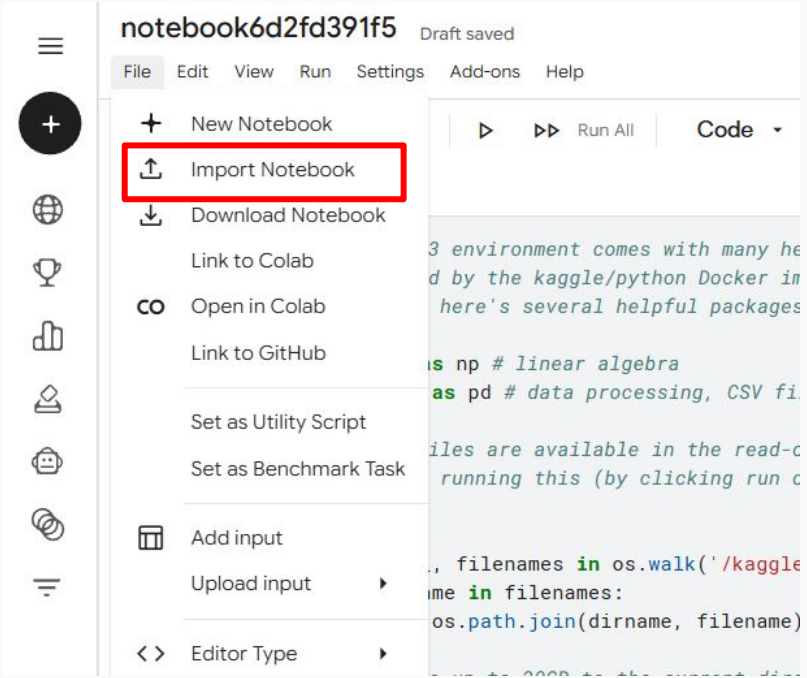
LINK: [NTHU NLP 2026] Assignment3: Fact Verification



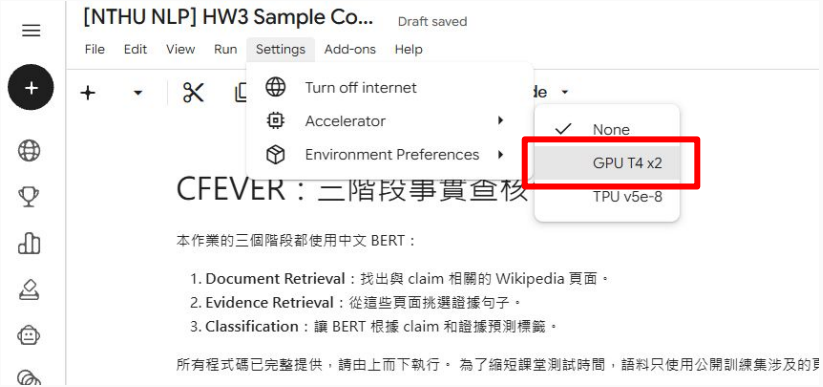
Create your code notebook



Upload the sample code



Run your code

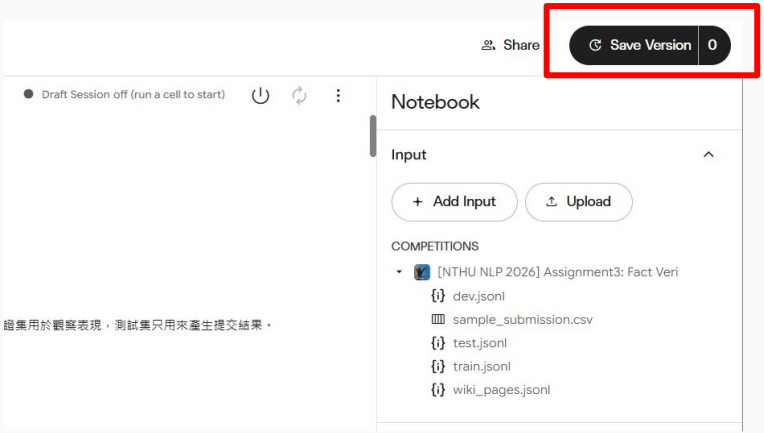


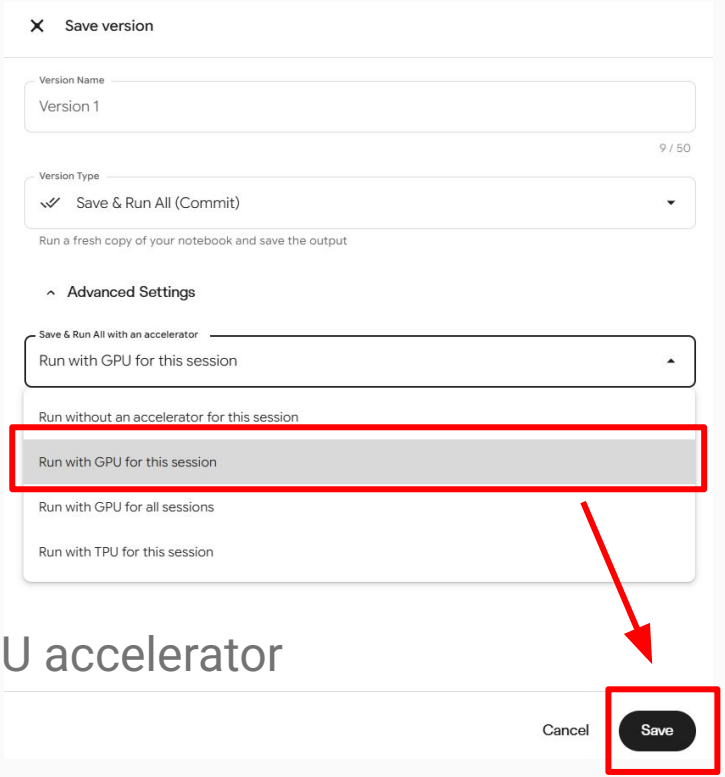
Enable the GPU accelerator before running your code.



Each account has a weekly GPU usage limit of 30 hours.

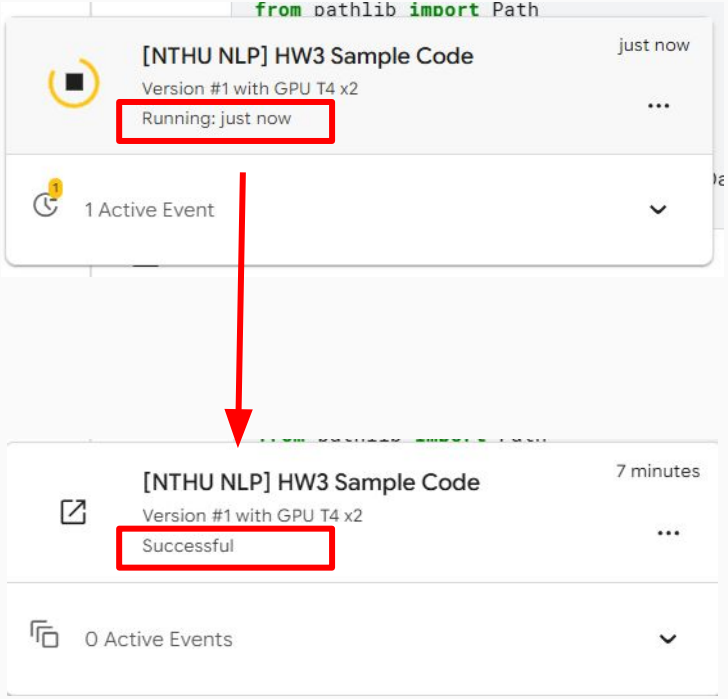
Submit your predictions

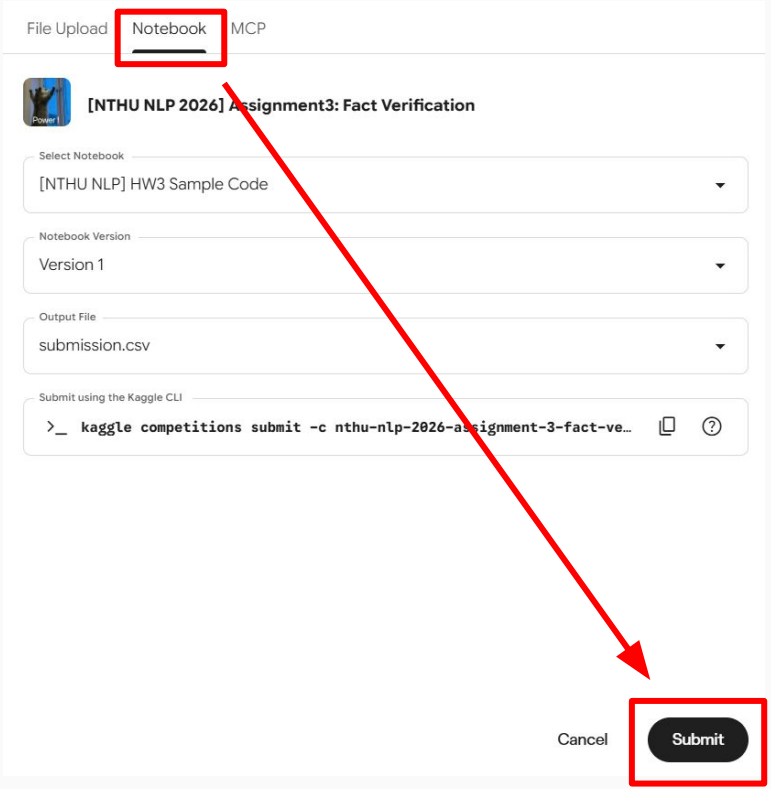




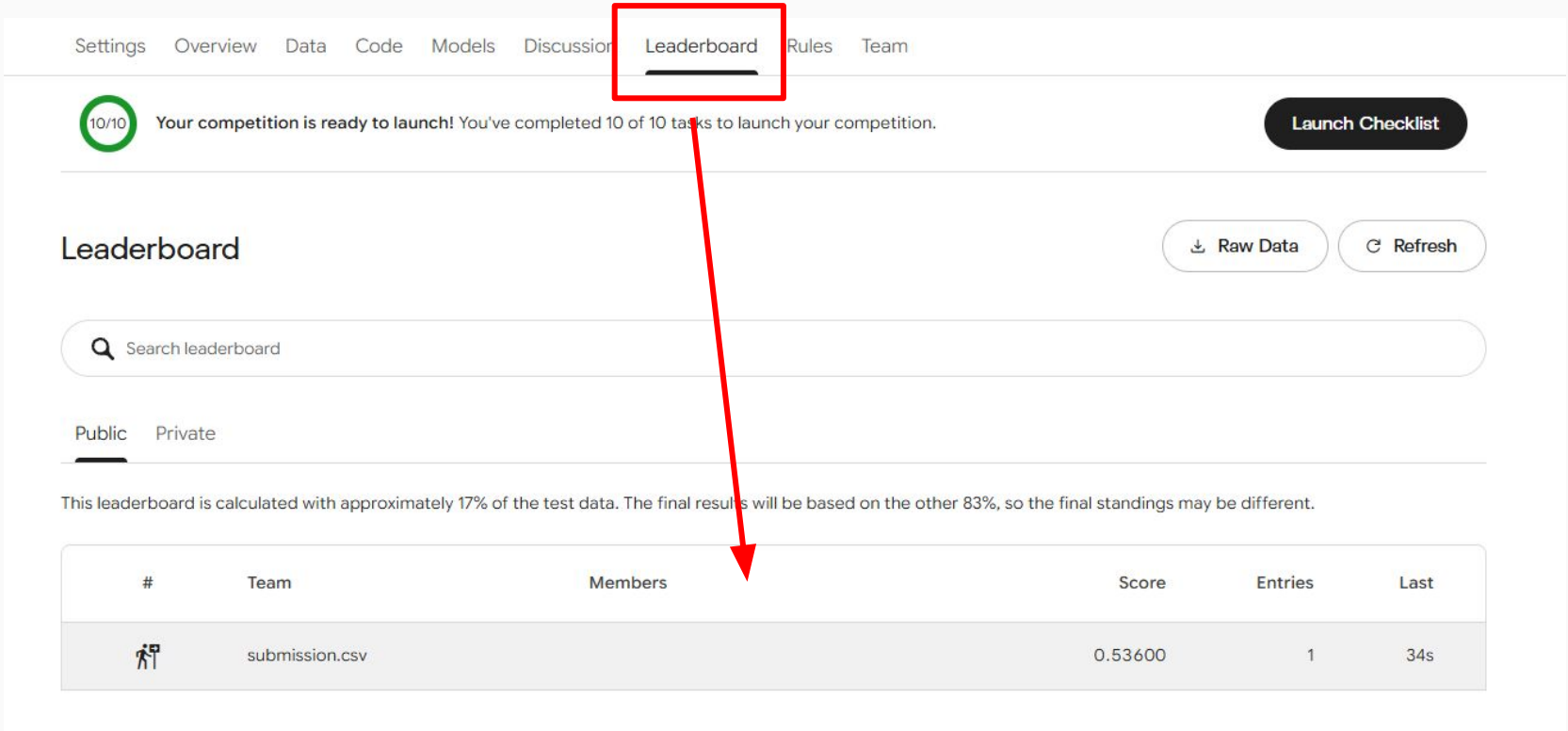
Turn-on the GPU accelerator

Submit your predictions





Leaderboard



Sample Code

TODO 1: Retrieve Wikipedia Documents

```
wanted_pages = set()
wanted_sentences = set()

for row in all_train_df.itertuples():
    # NOT ENOUGH INFO 沒有可用的標準證據句。
    if row.label.upper() == "NOT ENOUGH INFO":
        continue
    for group in row.evidence:
        for item in group:
            page_title = item[2]
            sentence_id = item[3]
            wanted_pages.add(page_title)
            wanted_sentences.add((page_title, sentence_id))

# Wikipedia 檔案較大，逐行讀取，僅保留需要的頁面。
wiki_pages = {}
with WIKI_PATH.open(encoding="utf-8") as file:
    for line in file:
        page = json.loads(line)
        if page["id"] in wanted_pages:
            wiki_pages[page["id"]] = page

print("候選頁面數: ", len(wiki_pages))
```

extract the page title and sentence id

read the wanted pates

TODO 1: Retrieve Wikipedia Documents

```python
def encode_texts(texts):
    """將一批文字轉為長度為 1 的 BERT 向量，回傳 CPU tensor。"""
    embeddings = []
    with torch.no_grad():
        for start in range(0, len(texts), EMBEDDING_BATCH_SIZE):
            batch_texts = texts[start:start + EMBEDDING_BATCH_SIZE]
            inputs = tokenizer(
                batch_texts, padding=True, truncation=True,
                max_length=RETRIEVAL_MAX_LENGTH, return_tensors="pt",
            )
            for name in inputs:
                inputs[name] = inputs[name].to(device)

            # token_vectors 的形狀是 [batch 大小, token 數, 768]。
            token_vectors = retrieval_model(**inputs).last_hidden_state
            mask = inputs["attention_mask"].unsqueeze(-1).float()

            # 只平均 attention_mask = 1 的 token；padding 的權重是 0。
            vector_sum = (token_vectors * mask).sum(dim=1)
            token_count = mask.sum(dim=1).clamp(min=1)
            text_vectors = vector_sum / token_count
            text_vectors = F.normalize(text_vectors, p=2, dim=1)
            embeddings.append(text_vectors.cpu())

    return torch.cat(embeddings, dim=0)
```

tokenize the input sequence

input the sequence into the model

normalize the vector for cosine similarity

construct sequence embedding matrix

TODO 1: Retrieve Wikipedia documents

```python
def retrieve_documents(claim_embeddings, top_k):
    """每個 claim 回傳一份頁面名稱清單，順序為相似度由高到低。"""
    similarities = claim_embeddings @ document_embeddings.T
    count = min(top_k, len(page_titles))
    top_indices = similarities.topk(k=count, dim=1).indices.tolist()

    results = []
    for indices in top_indices:
        pages = []
        for index in indices:
            pages.append(page_titles[index])
        results.append(pages)
    return results

train_claim_embeddings = encode_texts(train_df["claim"].tolist())
dev_claim_embeddings = encode_texts(dev_df["claim"].tolist())
test_claim_embeddings = encode_texts(test_df["claim"].tolist())

train_documents = retrieve_documents(train_claim_embeddings, DOCUMENT_TOP_K)
dev_documents = retrieve_documents(dev_claim_embeddings, DOCUMENT_TOP_K)
test_documents = retrieve_documents(test_claim_embeddings, DOCUMENT_TOP_K)

print("測試 claim: ", test_df.loc[0, "claim"])
print("取回頁面: ", test_documents[0])
```

compute similarities

select topk

TODO 2: Retrieve the Evidence

```python
sentence_info = []      # 第 i 個元素是 (頁面名稱, 原始句子編號)。
sentence_texts = []     # 第 i 個元素是對應句子的文字。

for title in page_titles:
    for line in wiki_pages[title]["lines"].splitlines():
        parts = line.split("\t")
        if len(parts) < 2 or not parts[0].isdigit():
            continue
        sentence_id = int(parts[0])
        text = parts[1].strip()
        if text and (title, sentence_id) in wanted_sentences:
            sentence_info.append((title, sentence_id))
            sentence_texts.append(text)

sentence_embeddings = encode_texts(sentence_texts)
print("候選證據句數: ", len(sentence_info))
print("句子向量矩陣大小: ", sentence_embeddings.shape)
```

split the ids and sentencess

load the wanted sentence

TODO 2: Retrieve the Evidence

```python
def retrieve_evidence(claim_embeddings, document_results, top_k):
    """回傳證據座標，以及給 BERT 閱讀的句子文字。"""
    all_evidence = []
    all_contexts = []

    for claim_vector, pages in zip(claim_embeddings, document_results):
        # 只允許從第一階段取回的頁面選句。
        candidate_indices = []
        for index, (title, sentence_id) in enumerate(sentence_info):
            if title in pages:
                candidate_indices.append(index)

        evidence = []
        selected_texts = []
        if candidate_indices:
            # claim 與候選句都已正規化，所以內積就是 cosine similarity。
            candidates = sentence_embeddings[candidate_indices]
            scores = candidates @ claim_vector
            count = min(top_k, len(candidate_indices))
            ranked_indices = scores.topk(k=count).indices.tolist()

            for local_index in ranked_indices:
                index = candidate_indices[local_index]
                title, sentence_id = sentence_info[index]
                evidence.append([title, sentence_id])
                selected_texts.append(sentence_texts[index])

        all_evidence.append(evidence)
        all_contexts.append(" ".join(selected_texts)[:1000])
    return all_evidence, all_contexts

train_evidence, train_contexts = retrieve_evidence(train_claim_embeddings, train_documents, EVIDENCE_TOP_K)
dev_evidence, dev_contexts = retrieve_evidence(dev_claim_embeddings, dev_documents, EVIDENCE_TOP_K)
test_evidence, test_contexts = retrieve_evidence(test_claim_embeddings, test_documents, EVIDENCE_TOP_K)

print("證據座標: ", test_evidence[0])
print("證據文字: ", test_contexts[0])
```

compute similarities

select topk

TODO 3: Claim Verification

Prepare dataset

```
labels = ["supports", "refutes", "NOT ENOUGH INFO"]
label_to_id = {"SUPPORTS": 0, "REFUTES": 1, "NOT ENOUGH INFO": 2}
train_labels = train_df["label"].str.upper().map(label_to_id).tolist()

class ClaimDataset(Dataset):
    def __init__(self, claims, contexts, target_labels=None):
        # 固定長度 padding，讓 DataLoader 可以直接把多筆資料堆成一個 batch。
        self.encodings = tokenizer(
            list(claims), contexts, truncation=True,
            padding="max_length", max_length=MAX_LENGTH,
        )
        self.labels = target_labels

    def __len__(self):
        return len(self.encodings["input_ids"])

    def __getitem__(self, index):
        item = {}
        for name, values in self.encodings.items():
            item[name] = torch.tensor(values[index], dtype=torch.long)
        if self.labels is not None:
            item["labels"] = torch.tensor(self.labels[index], dtype=torch.long)
        return item

train_dataset = ClaimDataset(train_df["claim"], train_contexts, train_labels)
dev_dataset = ClaimDataset(dev_df["claim"], dev_contexts)
test_dataset = ClaimDataset(test_df["claim"], test_contexts)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
dev_loader = DataLoader(dev_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)
```

Prepare random seed, model, and optimizer

```
# 比較不同檢索方法時，使用相同的分類器初始化。
torch.manual_seed(SEED)
model = AutoModelForSequenceClassification.from_pretrained(
    str(MODEL_DIR), num_labels=3, local_files_only=True,
    id2label=dict(enumerate(labels)),
    label2id={label: index for index, label in enumerate(labels)},
)
model = model.to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)
```

TODO 3: Claim Verification

```
training_losses = []
for epoch in range(EPOCHS):
    model.train()
    total_loss = 0

    for step, batch in enumerate(train_loader, start=1):
        # 將這個 batch 的每個 tensor 搬到 GPU (或 CPU)。
        for name in batch:
            batch[name] = batch[name].to(device)

        optimizer.zero_grad()
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()

        total_loss += loss.item()
        if step % 50 == 0 or step == len(train_loader):
            print(f"Epoch {epoch + 1}, step {step}, average loss = {total_loss / step:.4f}")

    training_losses.append(total_loss / len(train_loader))

model.save_pretrained(OUTPUT_DIR / "finetuned_bert")
tokenizer.save_pretrained(OUTPUT_DIR / "finetuned_bert")
```

fine-tune the model

Some Improving method

Here are some possible approaches to improving your CFEVER system

Models/Pipeline:

1. Use more powerful pretrained language models for claim verification.
2. Explore more effective retrieval methods, such as dense retrieval or hybrid retrieval.
3. Use reranking methods to improve retrieval accuracy.
4. Use model ensembling to improve prediction accuracy.

Data:

5. Apply data augmentation to improve model generalization.
6. Use hard negative mining to improve retrieval performance.
7. Apply query expansion to improve document retrieval.
8. Combine multiple evidence sentences for more accurate claim verification.

Training:

9. Fine-tune retrieval models for document retrieval and evidence retrieval.
10. Tune hyperparameters to optimize model performance.

Scoring for TODOs

TODO	Score
TODO 1: Retrieve Wikipedia Documents	10%
TODO 2: Retrieve the Evidence	10%
TODO 3: Claim Verification	10%

The Kaggle competition starts on November 4, 2026, at 9:00 AM and ends on November 23, 2026, at 12:00 PM.

The private leaderboard scores will be released on November 23, 2026, at 12:00 PM.

Submission

Scoring

Coding work : 30% (10% for each of the Three TODOs)

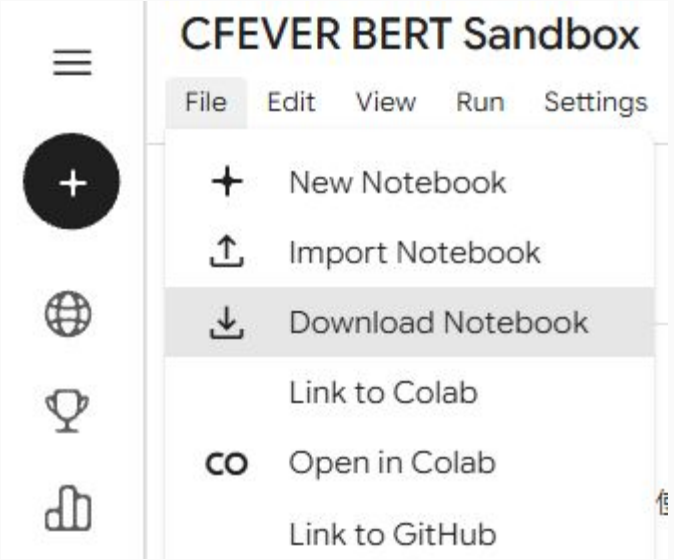
Leaderboard: 30% (Higher private scores earn more points, while both public and private scores are taken into account.)

Report: 40%

- Kaggle Username.
- Public & Private scores.
- Describe your method for retrieving Wikipedia documents (5%) and report Recall@N on development set (5%).
- Describe your method for retrieving evidence (5%) and report Recall@N on development set (5%).
- Describe your classification method (5%) and report Accuracy on development set (5%).
- **… Anything that can strengthen your report.** (10%)
- Generative AI Usage Statement.

Delivery policies: File formats

- Coding work: Python file (.ipynb)
  - Download your script via Colab/Kaggle.
- No other formats are allowed.
- Zip the files above before uploading you assignment.



Delivery policies: Filenames

	Filename rule	**Filename example**
Coding work	NLP_HW3_school_student_ID.ipynb	NLP_HW3_NTHU_12345678.ipynb
Zipped file	NLP_HW3_school_student_ID.zip	NLP_HW3_NTHU_12345678.zip

Delivery policies: Rules of coding

- If you use ChatGPT or Generative AI, please specify your usage **both** in:
  - **Code comments**
  - **Reports**
- **No plagiarism**. You should not copy and paste from your classmates. **Submit duplicate code or report will get 0 point !**
- Please provide links if you take the code from the Internet as reference.
- The following behaviors will cause loss in the score of the assignment: **(1) Usage with Generative AI without specifications (2) Internet sources without specifications (3) Plagiarism.**

Punishments

Rule	Name your code: NLP_HW3_school_student_ID.ipynb (only .ipynb is acceptable)	Name your file: NLP_HW3_school_student_ID.zip	Provide your Kaggle username
Punishment	-5	-5	-10
Rule	Report your generative AI usage	Your code or report should not shows a high degree of similarity to another student's submission.	
Punishment	-10	-100 for both	

Uploading the zipped file

- Please upload your file to NTU COOL.
- You will have three weeks to finish this assignment.
- If you have any question, please e-mail to **nthuikmlab@gmail.com**